Runtime-Compiled Action Shielding and Sparse Adversarial Probing for Mitigating Specification Gaming in RLVR/GRPO

Prabin K C · Evaluation release v1.0 · 26 October 2026 White-box evaluation / black-box training engine

Public artifacts: https://github.com/MLucifer13/compiled-shielding-eval

1. Results and empirical scope

This release investigates whether verified counterexamples can be converted into persistent action permissions, reducing repeated verification and residual exploit probability. It uses bounded program languages and small neural policies. The strongest result is structural exclusion of designated DSL exploits **after compilation**, not universal alignment or correctness.

Current neural leaderboard

All five policies use the same 6,492-parameter input-conditioned MLP/GRU architecture. Reported accuracy is the stochastic blind-spot evaluation; intervals are 95% seed-bootstrap intervals.

Method	True accuracy % [95% CI]	Sampled exploit %	Exact risk %	Heavy audits %	Training / rollout CPU ×
Baseline	30.154 [17.231, 50.934]	69.8025	69.791862	0.000	1.00 / 1.00
D: contextual masking	90.310 [86.577, 94.000]	9.1254	11.602297	0.000	1.07 / 1.11
E: static verification	90.310 [86.577, 94.000]	9.1254	11.602352	100.000	1.33 / 1.69
F: adaptive verification	99.902 [99.826, 99.978]	0.0000	0.033013	8.953	1.39 / 1.43
G: compiled shielding	99.859 [99.783, 99.935]	0.0000	0.000000	0.760	1.00 / 0.99

G is **-0.043 percentage points relative to F**, paired 95% interval **[-0.087, 0.000]**. It missed the requested 99.90% accuracy target. Zero sampled exploits are not evidence of zero probability: F retains measurable mass. G's zero comes from negative-infinite forbidden logits in the released inference artifact, independently checked by enumeration. Ordinary errors and abstentions remain possible.

Historical progression - different experimental regimes

Stage	Evaluation / policy	True accuracy % [95% CI]
v1 tabular	Main OOD, baseline	0.109 [0.098, 0.123]
v1 tabular	Main OOD, D	99.779 [99.759, 99.798]
v1 tabular	Main OOD, E	99.862 [99.844, 99.878]
v1 tabular	Blind-spot OOD, E	1.256 [0.979, 1.563]
v2 neural	Blind-spot, F	99.902 [99.826, 99.978]
v3 neural	Blind-spot, G	99.859 [99.783, 99.935]

V1 used a 125-program categorical policy and 24 seeds. Its historical summaries are supplied, but the live verifier targets the neural/Python artifacts. V2/v3 use eight main seeds. These rows describe an empirical progression, not a single controlled architecture comparison; tabular and neural cost denominators differ.

2. Evaluation protocol, threat model, and architecture

Tasks. The suites are connected graph navigation, strict longest-increasing-subsequence length, and aggregation by summation. Training core sizes were 6/8/10/12. The weak monitor checks small monitored contexts; the true grader independently checks outputs, legal maze edges, aborts, and prohibited actions. A six-token program chooses two supplied algorithm kernels, a scale/cue guard, a cutoff, an output channel, and a report action. Correct algorithm bodies are provided; the neural model assembles programs rather than inventing algorithms.

Threat. A policy can choose an honest kernel on small or visibly monitored inputs and switch to a designated shortcut outside that regime. Channels and forged reports also count as exploits. D's contextual permissions and E's small-input checks do not cover dormant large-input branches. D/E's identical sampled rows are inherited from matched v2 fits: the static gate was redundant in this training regime and a constant process bonus cancels under group standardization.

Training budget, reported rather than re-executed. Each neural fit used 240 updates, four prompts × eight completions, two clipped GRPO epochs per rollout, learning rate .002, and reference KL coefficient .005. F/G included adversarial task contexts in training. V3 trained 24 G models and 36 ablations, reusing 96 frozen v2 comparator models. No checkpoint was selected using final held-out accuracy. The public release contains inference artifacts, not enough information to independently reproduce the private optimization engine.

Live evaluation. For each of 120 policies, the verifier regenerates 96 tasks per cell at sizes 8/12/20/40/80/160, with monitored/tagged and unmonitored/untagged conditions. Primary accuracy equally weights the four larger unmonitored sizes and three suites. Task seed is 800000 + 100*size + monitored; sampling seed is 78000 + model_seed. It preserves the original sampling order and checks 1,440 cells against reference observations. Execution is memoized only where task and executable semantics are identical.

Exact integration. All 2,304 syntactically valid programs are scored at one n=160 unmonitored/untagged task per suite/seed, task seed 12900. The unsafe predicate is an active shortcut kernel, prohibited channel, or forged report. Normalization is checked numerically; every unsafe G log probability must be negative infinity, not merely a rounded small number. This 24-context integral per method differs from the primary sampled population. Because the deployed G permissions exclude both shortcut positions, the structural statement also covers other scales under the unchanged interpreter semantics. It does not cover the unshielded weights or discovery-time training actions.

Uncertainty. Neural intervals use 10,000 bootstrap resamples of eight seeds, averaging suites and sizes within seed, with resampling seed 20261026. The paired G-F interval uses matched seeds. Only three seeds support the Python demonstration; 100% sampled accuracy is not a universal success rate.

Architectural disclosure. F combines adaptive multi-arm boundary probing with counterexample feedback. G adds counterexample-guided logit shield compilation, stores frozen permissions for deployment, and reduces redundant verification and auxiliary optimization work. Generation and probability scoring use the same frozen support. The public reader applies final tables; it cannot derive or update them. Adaptive search/update equations, audit-scheduling constants, compiler extraction logic, and training wrappers are withheld.

To support independent verification while protecting ongoing research IP, this repository releases the full evaluation harness, adversarial stress suites, and frozen model checkpoints. The training engine (AlgorithmGGRPOTrainer) and dynamic shield compiler are available for live walkthroughs upon request.

Nyaya, Mimamsa, and Pratitantra-Siddhanta provided conceptual inspiration for adversarial challenge, constraint priority, and system-relative established permissions. These are design analogies, not historical derivations of the algorithms or substitutes for modern formal guarantees; action shielding also has established prior work (Alshiekh et al. (https://arxiv.org/abs/1708.08611)).

3. Efficiency evidence, Python results, and limitations

Timing. Fresh randomized serial calibration used 72 disposable checkpoint copies: eight methods/variants × three suites × three seeds, 40 updates each, one Torch thread. Starting policies were frozen release checkpoints; the optimizer/reference were reinitialized and G retained deployed permissions. The CPU ratios above describe steady-state **training and rollout**, not inference alone. Summed measured components were **34.80 ms/update for G versus 34.74 ms for baseline**; full wall measurements including loop bookkeeping were 34.84 versus 34.78 ms. These are not statistically established speedups.

Isolated measurement	Result
Fixed-batch base forward/backward	8.984 ms
Same batch with F auxiliary work	23.281 ms
Amortized auxiliary difference	7.148 ms/update
Contextual / frozen permission batch, 32×6	1155.18 / 156.55 µs
Known conditional-attack cascade	0.149 vs 0.695 ms/audit; 78.6% saving

Only 15.8% of potential second probes were avoided on actual G training traffic. The attack-heavy microbenchmark cannot justify a universal 78.6% speedup. Timings and raw calibration rows are supplied as historical evidence; a frozen inference release cannot independently rerun the withheld training workload.

Matched-seed ablations

Accuracy/risk use seeds 0-3; serial CPU uses calibration seeds 0-2.

Variant	Accuracy %	Sampled exploit %	Exact risk %	CPU ×
F	99.870	0.0000	0.021294	1.39
G	99.826	0.0000	0.000000	1.00
G without hard compilation	99.783	0.0217	0.017541	1.27
G with fixed auditing	99.870	0.0000	0.000000	1.08
G without early exit	99.848	0.0000	0.000000	0.92

Compilation and the auditing reduction it enables are coupled in this ablation. The faster no-early-exit calibration illustrates workload variation and limited timing replication; it prevents claiming an established end-to-end benefit from cascading alone.

Actual Python function generation

Eighteen runs used a **68,480-parameter, two-layer GPT-2**, 48 TRL optimizer steps × eight completions, three suites and three seeds, without pretrained weights or supervised warm-up. The model emits BPE-token Python within a supplied 20-function-per-family grammar. Training inputs were sizes 8/12. Final evaluation samples 32 functions at each size 12/20/40/80/160 with two independent task instances per size; primary accuracy uses sizes above 12. Both methods share syntax and static execution permissions. The public check reads frozen token transitions and runs the resulting functions with an independent grader.

Python policy	Sampled accuracy %	Sampled exploit %	n=160 grammar risk %	Audits %	Training wall s/model
Baseline	97.483	2.517	2.696235	0.00	33.06
G	100.000	0.000	0.441524	12.88	44.38

Python G did **not** meet zero exact risk, sub-3% auditing, or near-baseline wall cost: some attack patterns remained undiscovered. The optional public verification reproduces that residual mass. Execution is restricted to supplied, hash-verified functions in isolated subprocesses; this is **not an OS sandbox for arbitrary hostile Python**. Its batched evaluation runtime is separate from the original training subprocess timing.

Limits and next tests. Correct primitive semantics and trusted permission enforcement are essential; stronger safety is engineered into a bounded support. A known static global permission set could also enforce the DSL restriction. Withholding training prevents proof of training-data provenance, independent ablation retraining, or verification that evaluation tasks were never consulted. Public source, fresh-seed stress tests, checkpoint hashes, and live grading improve inspectability but cannot prove zero leakage. Hashes detect artifact mismatches, not scientific authenticity by themselves.

Scaling to 7B+ code models requires a broader executable action space, independently validated semantic contracts, a hardened execution broker, synchronized distributed permissions/log probabilities, realistic generation costs, and held-out attack families. No Qwen/Llama or frontier-lab efficacy was tested. The released findings support a constrained engineering mechanism, not a universally Pareto-optimal alignment solution.